

Contedores

Índice

- Motivación e historia
 - Vantaxes
 - Inconvenientes
 - Ciberseguridade e contedores
 - Tecnoloxías actuais
 - Comparativa Docker/Podman
-

Motivación e Historia

- Resolver incompatibilidades entre contornos.
 - **Primeiros pasos:**
 - *chroot* (1979)
 - *FreeBSD Jails* (2000)
 - *OpenVZ* (2005)
 - *LXC* (2008)
 - *LXD* (2015)
 - **Docker** populariza os contedores en 2013.
-

Vantaxes dos contedores

- Lixeireza e velocidade
 - Portabilidade entre contornos
 - Illamento de procesos
 - Escalabilidade e integración con CI/CD
-

Inconvenientes dos contedores

- Menor illamento cós hipervisores
 - Posibles riscos de seguridade
 - Xestión complexa de volumes e persistencia
 - Dependencia do kernel Linux
-

Ciberseguridade e contedores

Riscos

- Container escape
- Imaxes inseguras
- Permisos excesivos

Medidas

- Escaneo con **Trivy, Clair**
- Execución **sen root**
- Uso de **AppArmor, SELinux**

Tecnoloxías actuais

- **Docker:** contedores lixeiros e populares
- **Podman:** alternativa sen *daemon*
- **LXC / LXD:** máis preto dunha VM
- **Kubernetes:** orquestración
- **containerd, CRI-O:** motores backend

Comparativa Docker vs Podman

Aspecto	Docker	Podman
Daemon	Precisa daemon	Non require daemon
Execución	Execútase como servizo de fondo	Execútase directamente como proceso
Seguridade	Require permisos elevados	Pode executarse como usuario sen root
Compatibilidade	Moi estendido	Compatible con imaxes e comandos de Docker
Integración	Docker Compose, Swarm	Podman Compose, integración con Kubernetes

Conceptos básicos

Contedor

Proceso empaquetado executándose illado do resto de procesos:

- Instancia executable dunha **imaxe**.
- Pode **crear, iniciar, deter, mover ou eliminarse** por liña de comandos, GUI ou API.
- Execútase en **local ou na nube**.
- É **portable** e illado doutros contedores e procesos.

Imaxe

Modelo para crear contedores.

- Emprega un sistema de ficheiros illado.
- Poden:
 - **Descargarse dun repositorio (DockerHub)**
 - **Definirse a partir dun Dockerfile / Containerfile**

Docker

Instalación en Windows

1. Instalar **WSL**

```
wsl --install -d distribución  
# Exemplo:  
wsl --install -d ubuntu
```

2. Instalar **Docker Desktop**

3. Engadir o usuario ao grupo **docker-users**

Para engadir o usuario ao grupo:

Administración de Equipos → Usuarios e grupos locais → Grupos → docker-users

Instalación en Ubuntu

1. Instalar paquetes necesarios e claves:

```
sudo apt-get update  
sudo apt-get install ca-certificates curl gnupg lsb-release  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o  
/usr/share/keyrings/docker-archive-keyring.gpg
```

2. Engadir o repositorio oficial:

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-  
archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs)  
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

3. Instalar Docker e engadir o usuario ao grupo:

```
sudo apt-get update  
sudo apt-get install docker-ce docker-ce-cli containerd.io  
sudo usermod -aG docker $USER
```

Verificar a instalación

```
docker run hello-world
```

Se aparece:

"Hello from Docker! This message shows that your installation appears to be working correctly..."

...significa que Docker está correctamente instalado.

Docker: comandos básicos

Comandos sobre imaxes

- Descargar unha imaxe:

```
docker pull nome_imaxe
```

- Ver imaxes descargadas:

```
docker image ls
```

- Eliminar unha imaxe:

```
docker image rm nome_imaxe
```

Comandos sobre contedores

- Executar un contedor:

```
docker run nome_imaxe
```

- En segundo plano:

```
docker run -d nome_imaxe
```

- Especificando nome:

```
docker run --name nome_contedor nome_imaxe
```

- Redirección de portos:

```
docker run -p porto_host:porto_contedor nome_imaxe
```

Xestión de contedores

- Amosar contedores en execución:

```
docker ps [-a] [-q]  
# -a → todos os contedores, -q → só ID
```

- Arrincar ou parar contedores:

```
docker start nome_contedor  
docker stop nome_contedor
```

- Eliminar contedores:

```
docker rm [-f] nome_contedor  
# -f → forzar
```

Comandos sobre redes

- Amosar redes:

```
docker network ls
```

- Crear ou eliminar:

```
docker network create nome_rede  
docker network rm nome_rede
```

- Inspeccionar rede:

```
docker network inspect nome_rede
```

Volumes

Tipos:

- **Volumes de Docker:** xestionados por Docker, almacenados en `/var/lib/docker/volumes/`.
- **Bind mounts:** montan un directorio do sistema anfitrión no contedor.
- **tmpfs:** montaxe temporal en RAM (só en Linux).

Comandos:

- Crear ou eliminar:

```
docker volume create nome_volume  
docker volume rm nome_volume
```

- Inspeccionar:

```
docker volume inspect nome_volume
```

- Empregar un volume:

```
docker run -v nome_volume:ruta_no_contedor nome_imaxe
```

- Empregar un bind mount:

```
docker run -v ruta_anfitrión:ruta_no_contedor nome_imaxe
```

Dockerfile (Creación de imaxes)

Un **Dockerfile** (ou *Containerfile*) é un ficheiro de texto que define como crear unha imaxe.

Instrucións habituais:

- **FROM:** imaxe base
- **RUN:** executar comandos
- **MAINTAINER:** autor
- **ENV:** variables de entorno
- **EXPOSE:** portos
- **ADD:** copiar arquivos
- **ENTRYPOINT:** comando principal

Exemplo:

```
FROM debian  
RUN apt-get update && apt-get install -y apache2 && apt-get clean && rm -rf
```

```
/var/lib/apt/lists/*
ENV APACHE_RUN_USER www-data
ENV APACHE_RUN_GROUP www-data
ENV APACHE_LOG_DIR /var/log/apache2
EXPOSE 80
ADD ["index.html", "/var/www/html/"]
ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
```

Crear unha imaxe

```
docker build -t imaxe:tag .
# 0 "." indica o directorio actual
```

Docker Compose

Ferramenta para despregar varios contedores de forma automatizada.

Estrutura do ficheiro **docker-compose.yml**

- **services:** definición de cada contedor
- **volumes:** volumes compartidos
- **networks:** redes personalizadas

Parámetros comúns en cada servizo:

- **image:** imaxe empregada
- **container_name:** nome do contedor
- **hostname:** nome do host
- **environment:** variables de entorno
- **ports:** port-forwarding
- **volumes:** volumes e bind mounts
- **network:** rede á que se conecta
- **command:** comando alternativo ao da imaxe

Comandos Docker Compose

Lanzar contedores

- **Arrancar todos os servizos definidos no **docker-compose.yml** do directorio actual.**

```
docker compose up          # arrinca en primeiro plano, amosando logs
docker compose up -d       # arrinca en segundo plano (detached)
docker compose -f arquivo.yml up # especifica un ficheiro concreto
```

- **Opcións moi prácticas ao arrancar:**

```
docker compose up --build          # recompila as imaxes se fixo cambios no
Dockerfile
docker compose up --pull always    # forza a descarga de imaxes máis recentes
docker compose up --no-deps servizo # arrinca un servizo sen levantar as
dependencias
docker compose up --scale web=3    # escala o servizo 'web' a 3 réplicas
```

Parar e limpar

- **Deter e borrar os contedores creados por compose (mantén volumes por defecto):**

```
docker compose down
docker compose -f arquivo.yml down
```

- **Limpezas máis agresivas:**

```
docker compose down -v          # elimina tamén os volumes anónimos
docker compose down --rmi local  # elimina imaxes creadas localmente
docker compose down --remove-orphans # elimina contedores orfos (de servizos
xa non definidos)
```

Estado e xestión básica

- **Ver que está en marcha e o seu estado:**

```
docker compose ps              # lista contedores do proxecto
```

- **Arrancar/parar só algúns servizos:**

```
docker compose start servizo1 servizo2
docker compose stop  servizo1 servizo2
```

- **Reiniciar servizos (útil tras cambios en variables de entorno):**

```
docker compose restart          # todos
docker compose restart servizo  # só un
```

Execución dentro de contedores

- **Entrar ou executar un comando nun servizo:**

```
docker compose exec servizo bash # shell interactiva se o contedor a ten instalada
docker compose exec servizo sh   # alternativa lixeira
docker compose exec servizo env   # inspeccionar variables de entorno
```

- **Crear e executar un contedor “one-off” (non definido en YAML):**

```
docker compose run --rm servizo comando # contedor efémero, ideal para scripts/migracións
```

Logs e diagnóstico

- **Ver logs con filtros útiles:**

```
docker compose logs          # logs de todos os servizos
docker compose logs servizo   # logs dun servizo
docker compose logs -f --tail=100 # seguir en tempo real, últimos 100 liñas
docker compose logs --since=10m  # últimos 10 minutos
```

- **Procesos en execución por servizo (debug):**

```
docker compose top          # mostra procesos (PID, comando...) por contedor
```

Compilación e imaxes

- **Construír imaxes definidas en `build::`**

```
docker compose build          # constrúe todas as imaxes
docker compose build servizo   # só unha
docker compose build --no-cache # evita usar cache
```

- **Xestionar imaxes remotas:**

```
docker compose pull          # baixa imaxes dos servizos
docker compose push          # sobe imaxes (se está configurado)
```

Validación e configuración

- **Validar e visualizar a configuración efectiva (incluíndo `.env`, overrides, etc.):**

```
docker compose config          # amosa YAML combinado e validado
docker compose config --services  # lista nomes de servizos
docker compose config --profiles  # lista perfís dispoñibles (se os hai)
```

Proxectos, perfís e múltiples ficheiros

- **Nome do proxecto (prefixo de contedores e redes):**

```
docker compose -p meu_proxecto up  # define o nome do proxecto
```

- **Usar varios ficheiros (override por ambiente):**

```
docker compose -f compose.yml -f compose.prod.yml up -d
```

- **Activar perfís (servizos opcionais):**

```
COMPOSE_PROFILES=monitorizacion,debug docker compose up -d
```

Copias e ficheiros

- **Copiar ficheiros entre host e contedor (cando o necesitas puntualmente):**

```
docker compose cp servizo:/ruta/no/contedor ficheiro_local
docker compose cp ficheiro_local servizo:/ruta/no/contedor
```

Diferencias con `docker logs` “clásico”

Se precisas os logs dun único contedor polo seu nome/ID (fóra do contexto de compose), tamén podes usar:

```
docker logs [-f] nome_contedor      # segue en tempo real os logs dese contedor
```

Pero, para proxectos compose, adoita ser máis cómodo empregar `docker compose logs`.

Exemplo

```

version: "3.9" # versión do formato de docker-compose

services:
  web:
    image: nginx:latest # Imaxe oficial de Nginx
    container_name: servidor_web
    ports:
      - "8080:80" # Porto 8080 do host → porto 80 do
    contedor
    volumes:
      - ./html:/usr/share/nginx/html # Monta o contido local en Nginx
    depends_on:
      - db # Espera a que a BD estea lista
    networks:
      - appnet

  db:
    image: mysql:8.0 # Imaxe oficial de MySQL
    container_name: base_datos
    restart: always # Reinicia automaticamente se falla
    environment:
      MYSQL_ROOT_PASSWORD: exemplo_root
      MYSQL_DATABASE: exemplo_db
      MYSQL_USER: usuario
      MYSQL_PASSWORD: contrasinal
    volumes:
      - db_data:/var/lib/mysql # Volume persistente
    networks:
      - appnet

volumes:
  db_data: # Define volume persistente para a BD

networks:
  appnet: # Rede interna entre web e BD

```

Referencias

- [Podman Docs](#)
- [Docker primeiros pasos](#)
- [Docker Compose](#)
- [Dockerfile Reference](#)